Referencial teórico

Fundamentos de engenharia de software e de gestão da produção aplicados ao sistema 3E Operações

Resumo. Este capítulo apresenta o referencial teórico que sustenta o desenvolvimento do 3E Operações, um sistema web para apoiar a operação de uma indústria metalúrgica a partir da leitura, somente leitura, do sistema comercial legado (Top Gerente). O texto articula cinco campos: engenharia de requisitos, processos e qualidade de software, arquitetura e modelagem de domínio, integração de sistemas e gestão da produção. Para cada campo, apresentam-se os conceitos consolidados na literatura e a forma como foram aplicados ao projeto, de modo a evidenciar a coerência entre a fundamentação teórica e as decisões técnicas adotadas.

Palavras-chave: engenharia de software; engenharia de requisitos; integração de sistemas; sistemas de execução de manufatura; acessibilidade.

---

1 Introdução

A indústria de transformação, em especial os segmentos de pequeno e médio porte, frequentemente opera com controles fragmentados: informações comerciais em um sistema de gestão, apontamentos de produção em planilhas e ordens de serviço em papel. Essa fragmentação dificulta a rastreabilidade, atrasa a comunicação entre setores e reduz a visibilidade do andamento dos pedidos. O sistema 3E Operações foi concebido para atuar nesse espaço, organizando o trabalho por setor produtivo (Corte e Dobra, Telhas e Revenda) a partir dos pedidos originados no sistema comercial legado, sem substituí-lo.

O desenvolvimento de um sistema dessa natureza mobiliza decisões que vão da especificação de requisitos à arquitetura, passando pela integração com um sistema legado, pela interface com usuários em ambiente fabril e pela gestão da produção propriamente dita. Este capítulo reúne os fundamentos teóricos de cada uma dessas frentes, organizados da seguinte forma: a Seção 2 trata da engenharia de requisitos; a Seção 3, dos processos de desenvolvimento; a Seção 4, dos testes e da qualidade; a Seção 5, da arquitetura e da modelagem de domínio; a Seção 6, da integração de sistemas; a Seção 7, da gestão da produção e dos sistemas de execução de manufatura; a Seção 8, da interface, usabilidade e acessibilidade; a Seção 9, da segurança; e a Seção 10, das métricas e da entrega contínua. A Seção 11 sintetiza a articulação entre os campos.

2 Engenharia de requisitos e especificação

A qualidade de um sistema de software depende, em larga medida, da qualidade de sua especificação de requisitos. Requisitos ambíguos, incompletos ou não testáveis estão entre as principais causas de retrabalho e de insucesso em projetos (SOMMERVILLE, 2019; PRESSMAN; MAXIM, 2021). Para reduzir essa ambiguidade, a literatura propõe notações estruturadas que restringem a forma de expressão dos requisitos a padrões verificáveis.

Entre essas notações, destaca-se o *Easy Approach to Requirements Syntax* (EARS), que orienta a redação de requisitos no formato "Quando , o sistema deve ", combinando palavras-chave que distinguem condições, eventos e estados (MAVIN et al., 2009). A normalização internacional ISO/IEC/IEEE 29148 complementa essa abordagem ao definir processos e características de requisitos de qualidade, como verificabilidade e rastreabilidade (ISO/IEC/IEEE, 2018).

No 3E Operações, cada funcionalidade foi especificada com critérios de aceitação em notação EARS, e cada critério foi associado a uma tarefa de implementação e a um teste correspondente. Essa prática produziu uma cadeia de rastreabilidade entre requisito, implementação e evidência de teste, permitindo verificar, de forma objetiva, que o que foi construído corresponde ao que foi especificado — requisito fundamental para a validação do sistema junto aos usuários.

3 Processos de desenvolvimento de software

Métodos iterativos e incrementais, consolidados a partir do final da década de 1990, propõem o desenvolvimento em ciclos curtos, com entregas frequentes de partes funcionais do sistema (BECK, 2000; SOMMERVILLE, 2019). Essa abordagem reduz o risco do projeto ao permitir ajustes contínuos com base em retorno real, em oposição a modelos sequenciais nos quais o produto só é avaliado ao final.

A garantia da qualidade, por sua vez, apoia-se na distinção clássica entre verificação (o produto foi construído corretamente?) e validação (o produto correto foi construído?). Um princípio recorrente é a separação entre quem produz e quem verifica, de modo que a avaliação não seja contaminada pela autoria. Estudos empíricos reunidos por Forsgren, Humble e Kim (2018) reforçam que equipes que entregam em incrementos pequenos e frequentes, com automação e verificação sistemáticas, obtêm melhor desempenho tanto em velocidade quanto em estabilidade.

O 3E Operações adotou um fluxo orientado por especificação, no qual cada incremento percorre as etapas de especificação, projeto, definição de tarefas, execução e verificação. A verificação foi conduzida por um agente independente do autor da implementação, com checagem critério a critério, o que materializa, no processo, o princípio de independência entre produção e verificação.

4 Testes e qualidade de software

O desenvolvimento guiado por testes (*Test-Driven Development* — TDD) propõe escrever o teste antes do código de produção, de modo que os testes funcionem como especificação executável e como rede de segurança para refatorações (BECK, 2002). A automação de testes permite executar verificações repetidamente e a baixo custo, favorecendo a evolução do sistema sem regressões.

Para manter os testes rápidos e determinísticos, a literatura recomenda isolar o código sob teste de dependências externas por meio de dublês de teste (*stubs*, *spies*, *fakes* e *mocks*), escolhidos conforme o objetivo do teste (FOWLER, 2007). Complementarmente, a medição da qualidade do código e do esforço de teste apoia-se em métricas bem estabelecidas (FENTON; BIEMAN, 2014).

No projeto, o TDD foi aplicado aos comportamentos críticos do domínio — cálculo de saldo, disponibilidade, tratamento de perdas e refugos, permissões, prazos e entregas. O acesso a dados foi isolado por meio de implementações falsas nos testes, mantendo o domínio independente do banco de dados e os testes executáveis sem infraestrutura externa. Testes de ponta a ponta, com automação de navegador, cobriram os fluxos principais de interface.

5 Arquitetura de software e modelagem de domínio

A arquitetura de um sistema define sua estrutura e as decisões que afetam sua evolução a longo prazo (BASS; CLEMENTS; KAZMAN, 2021). Desde Parnas (1972), sabe-se que a decomposição de um sistema deve basear-se no **ocultamento de informação**: cada módulo deve encapsular decisões que podem mudar, expondo apenas o necessário às demais partes. Esse princípio está na raiz das noções modernas de coesão e acoplamento.

A modelagem orientada a domínio (*Domain-Driven Design* — DDD) propõe concentrar a complexidade do negócio em um modelo rico, expresso na linguagem dos especialistas do domínio, isolando-o de detalhes tecnológicos (EVANS, 2003; VERNON, 2013). Em paralelo, o estilo arquitetural hexagonal, ou de portas e adaptadores, separa o núcleo da aplicação das tecnologias de entrada e saída por meio de interfaces bem definidas (COCKBURN, 2005); a *Clean Architecture* formaliza essa separação em camadas concêntricas (MARTIN, 2017). Padrões de projeto catalogados por Gamma et al. (1994) e padrões de acesso a dados documentados por Fowler (2002) complementam esse repertório.

O 3E Operações materializa esses princípios ao manter o domínio independente do *framework* e do banco de dados. A comunicação com a infraestrutura ocorre por portas e adaptadores, com uma implementação real (baseada em ORM) e uma implementação falsa para testes. A organização em módulos por área de negócio (auth, usuários, setores, pedidos, integração, produção, ocorrências,

expedição, prazos e indicadores) busca alta coesão interna e baixo acoplamento entre módulos, favorecendo a manutenibilidade.

6 Integração de sistemas e modernização de legados

Sistemas legados raramente podem ser substituídos por completo sem risco significativo. A estratégia conhecida como *Strangler Fig* propõe a modernização incremental: novas funcionalidades são construídas de forma paralela e independente ao legado, e comportamentos são gradualmente migrados, até que o sistema antigo possa ser reduzido (FOWLER, 2004). Trabalhar com segurança sobre código legado, por sua vez, exige compreensão de seu comportamento e a criação de pontos de intervenção controlados (FEATHERS, 2004).

Quando a integração ocorre entre sistemas distintos, os padrões de integração empresarial descrevem soluções para troca de mensagens, incluindo o tratamento de falhas e a **idempotência** — propriedade que garante que o reprocessamento de uma mesma mensagem não produza efeitos duplicados (HOHPE; WOOLF, 2003). Em sistemas distribuídos, o estilo arquitetural REST formaliza princípios de interação entre serviços por meio de recursos e operações bem definidas (FIELDING, 2000).

No contexto industrial, a norma ISA-95 (IEC 62264) define modelos de integração entre o nível de gestão empresarial e o nível de operações de manufatura (*Manufacturing Operations Management* — MOM, ou MES), estabelecendo terminologia e modelos de informação que orientam a troca de dados entre esses níveis (ANSI/ISA-95, 2010).

O 3E Operações adota a leitura **somente leitura** do sistema legado, por meio de um conector local que executa consultas parametrizadas e devolve um *payload* normalizado ao sistema. A importação de pedidos é assíncrona, disparada sob demanda pelo número do pedido, e idempotente: repetir a importação não duplica dados. O tratamento de falhas transitórias utiliza nova tentativa com espera progressiva (*backoff*), em linha com os padrões de integração confiável. O sistema ocupa, assim, a posição de camada de operações de manufatura, consumindo dados comerciais e organizando a execução produtiva.

7 Gestão da produção e sistemas de execução de manufatura

A gestão da produção fundamenta-se em conceitos como planejamento e controle da produção (PCP), no qual se distinguem quantidades planejadas, produzidas e pendentes, bem como o conceito de **disponível para promessa** (*available-to-promise*), que expressa a quantidade que pode ser comprometida com o cliente (VOLLMANN et al., 2005; CORRÊA; CORRÊA, 2012; SLACK; BRANDON-JONES; JOHNSTON, 2018). A literatura brasileira de PCP sistematiza esses conceitos no

contexto nacional (LUSTOSA et al., 2008). Perdas, refugos e retrabalhos são tratados como categorias próprias de análise, essenciais ao controle da qualidade e à medição de desempenho.

Esses conceitos foram incorporados ao modelo de domínio do 3E Operações: a quantidade pendente é dada por `solicitado − executado`; a quantidade disponível para entrega, por `executado − entregue`, em analogia ao *available-to-promise*. Perdas e refugos são registrados com motivo obrigatório, mas **não abatem a obrigação** de produzir a quantidade solicitada, preservando a distinção entre quantidade conforme e quantidade refugada. Prazos podem ser definidos por item e por setor, e indicadores consolidam a produção por setor e o cumprimento de prazos, oferecendo suporte ao PCP.

8 Interface, usabilidade e acessibilidade

A interface é o ponto de contato entre o sistema e seus usuários; em ambientes fabris, ela é utilizada de forma contínua, muitas vezes em dispositivos móveis e em condições adversas. As diretrizes de acessibilidade para conteúdo web (WCAG 2.1) estabelecem critérios testáveis organizados em quatro princípios — perceptível, operável, compreensível e robusto —, com níveis de conformidade A, AA e AAA (W3C, 2018). A norma ISO 9241-210 orienta o design centrado no ser humano, e a ISO 9241-110 define princípios de interação, como autodescrição, conformidade com as expectativas do usuário e tolerância a erros (ISO, 2019; ISO, 2020).

Complementarmente, as heurísticas de usabilidade de Nielsen oferecem critérios práticos de avaliação, entre os quais se destacam a visibilidade do estado do sistema, a prevenção de erros e a apresentação de mensagens de erro em linguagem clara (NIELSEN, 1994). No Brasil, a Norma Regulamentadora NR-17 estabelece parâmetros de ergonomia no trabalho, relevantes para interfaces de uso prolongado (BRASIL, 1978).

No projeto, a interface foi construída para funcionar de forma responsiva em celular, tablet, notebook e desktop, com alvo de conformidade WCAG 2.1 AA. Foram adotados foco visível, rótulos acessíveis, mensagens de erro e de sucesso claras e tamanhos de alvo adequados ao toque. Buscou-se, ainda, evitar a exposição de identificadores técnicos ao usuário, apresentando informações em linguagem do domínio — decisão alinhada à segunda heurística de Nielsen (correspondência entre o sistema e o mundo real).

9 Segurança de aplicações web

A segurança de aplicações web é tratada de forma sistemática pelo *Application Security Verification Standard* (ASVS), da OWASP, que reúne requisitos de verificação para autenticação, controle de acesso, sessão e criptografia, e pelo *Top 10* de riscos, que destaca, entre as falhas mais críticas, o controle de acesso quebrado e as falhas de autenticação (OWASP, 2021; OWASP, 2025).

Para o armazenamento de senhas, a literatura recomenda funções de derivação de chave resistentes a ataques com hardware dedicado. O algoritmo **scrypt** é uma função *memory-hard*, especificada na RFC 7914, projetada para elevar o custo de ataques de força bruta (PERCIVAL; JOSEFSSON, 2016).

No 3E Operações, a sessão é mantida em *cookie* HTTP *httpOnly* assinado, as senhas são armazenadas com scrypt e as autorizações são decididas no servidor por meio de uma matriz de permissões por perfil. O acesso ao sistema legado utiliza um usuário com privilégios mínimos (somente leitura), em conformidade com o princípio do menor privilégio.

10 Métricas, qualidade e entrega contínua

A avaliação da qualidade de produto pode ser estruturada pelo modelo da ISO/IEC 25010, que define características como adequação funcional, confiabilidade, usabilidade, eficiência de desempenho, manutenibilidade, portabilidade, compatibilidade e segurança (ISO/IEC, 2011). Métricas clássicas de software apoiam a análise de complexidade e de estrutura: a complexidade ciclomática de McCabe (1976) e a suíte de métricas para orientação a objetos de Chidamber e Kemerer (1994) são referências consagradas; Fenton e Bieman (2014) discutem o uso criterioso dessas medidas.

A entrega contínua, por sua vez, propõe automatizar a construção, o teste e a implantação, de modo a tornar as liberações frequentes e confiáveis (HUMBLE; FARLEY, 2010). Convenções de mensagens de *commit* (Conventional Commits) e versionamento semântico contribuem para um histórico legível e rastreável (CONVENTIONAL COMMITS, 2019; PRESTON-WERNER, 2010).

No projeto, adotou-se integração contínua com verificação estática de tipos, análise estática (*lint*), execução de testes e construção da aplicação. A complexidade e o acoplamento foram monitorados nos pontos de maior risco, e a cobertura de testes foi utilizada como indicador de qualidade nos módulos críticos de domínio. As mensagens de *commit* seguem a convenção de *commits* convencionais.

11 Síntese

O conjunto de fundamentos apresentado articula-se em torno de um princípio comum: a separação de responsabilidades. Na engenharia de requisitos, separa-se o que o sistema deve fazer (critérios testáveis) de como será feito; nos testes, separa-se a especificação executável da implementação; na arquitetura, separa-se o domínio da infraestrutura; na integração, separa-se o sistema novo do legado; e na gestão da produção, separa-se o nível de planejamento do nível de execução. É essa separação que confere ao 3E Operações a possibilidade de evoluir de forma incremental, de integrar-se a um sistema legado sem comprometê-lo e de apoiar a operação de

fábrica com rastreabilidade e segurança. O referencial teórico aqui reunido, portanto, não apenas justifica as escolhas técnicas do projeto, mas também oferece os critérios pelos quais sua qualidade pode ser avaliada.

Referências

ANSI/ISA-95.00.01. **Enterprise-Control System Integration — Part 1: Models and Terminology**. International Society of Automation, 2010.

BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 4. ed. Boston: Addison-Wesley, 2021.

BECK, K. **Extreme Programming Explained: Embrace Change**. Boston: Addison-Wesley, 2000.

BECK, K. **Test-Driven Development: By Example**. Boston: Addison-Wesley, 2002.

BRASIL. Ministério do Trabalho e Emprego. **Norma Regulamentadora n. 17 — Ergonomia (NR-17)**. Brasília, 1978.

CHIDAMBER, S. R.; KEMERER, C. F. A Metrics Suite for Object Oriented Design. **IEEE Transactions on Software Engineering**, v. 20, n. 6, p. 476-493, 1994.

COCKBURN, A. **Hexagonal Architecture (Ports and Adapters)**. 2005.

CONVENTIONAL COMMITS. **Conventional Commits 1.0.0**. 2019. Disponível em: https://www.conventionalcommits.org. Acesso em: 2026.

CORRÊA, H. L.; CORRÊA, C. A. **Administração de Produção e Operações**. 2. ed. São Paulo: Atlas, 2012.

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**. Boston: Addison-Wesley, 2003.

FEATHERS, M. **Working Effectively with Legacy Code**. Upper Saddle River: Prentice Hall, 2004.

FENTON, N.; BIEMAN, J. **Software Metrics: A Rigorous and Practical Approach**. 3. ed. Boca Raton: CRC Press, 2014.

FIELDING, R. T. **Architectural Styles and the Design of Network-based Software Architectures**. 2000. Tese (Doutorado) — University of California, Irvine, 2000.

FORSGREN, N.; HUMBLE, J.; KIM, G. **Accelerate: The Science of Lean Software and DevOps**. Portland: IT Revolution, 2018.

FOWLER, M. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

FOWLER, M. **Strangler Fig**. 2004. Disponível em: https://martinfowler.com/bliki/StranglerFigApplication.html. Acesso em: 2026.

FOWLER, M. **Mocks Aren't Stubs**. 2007. Disponível em: https://martinfowler.com/articles/mocksArentStubs.html. Acesso em: 2026.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. **Design Patterns: Elements of Reusable Object-Oriented Software**. Boston: Addison-Wesley, 1994.

HOHPE, G.; WOOLF, B. **Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions**. Boston: Addison-Wesley, 2003.

HUMBLE, J.; FARLEY, D. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**. Boston: Addison-Wesley, 2010.

ISO. **ISO 9241-210:2019 — Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems**. Genebra: ISO, 2019.

ISO. **ISO 9241-110:2020 — Ergonomics of human-system interaction — Part 110: Interaction principles**. Genebra: ISO, 2020.

ISO/IEC. **ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models**. Genebra: ISO, 2011.

ISO/IEC/IEEE. **ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering**. Genebra: ISO, 2018.

LUSTOSA, L. J.; MESQUITA, M. A.; QUELHAS, O. L. G.; OLIVEIRA, R. J. **Planejamento e Controle da Produção**. Rio de Janeiro: Elsevier, 2008.

MARTIN, R. C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Upper Saddle River: Prentice Hall, 2017.

MAVIN, A.; WILKINSON, P.; HARWOOD, A.; NOVAK, M. Easy Approach to Requirements Syntax (EARS). In: **17th IEEE International Requirements Engineering Conference (RE)**. Atlanta: IEEE, 2009.

McCABE, T. J. A Complexity Measure. **IEEE Transactions on Software Engineering**, v. SE-2, n. 4, p. 308-320, 1976.

NIELSEN, J. Heuristic Evaluation. In: NIELSEN, J.; MACK, R. L. (Ed.). **Usability Inspection Methods**. New York: John Wiley & Sons, 1994.

OWASP. **OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks**. 2021. Disponível em: https://owasp.org/Top10. Acesso em: 2026.

OWASP. **Application Security Verification Standard (ASVS) 5.0.0**. 2025. Disponível em: https://owasp.org/www-project-application-security-verification-standard. Acesso em: 2026.

PARNAS, D. L. On the Criteria To Be Used in Decomposing Systems into Modules. **Communications of the ACM**, v. 15, n. 12, p. 1053-1058, 1972.

PERCIVAL, C.; JOSEFSSON, S. **The scrypt Password-Based Key Derivation Function**. IETF RFC 7914, 2016. Disponível em: https://www.rfc-editor.org/rfc/rfc7914. Acesso em: 2026.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: uma abordagem profissional**. 9. ed. Porto Alegre: AMGH, 2021.

PRESTON-WERNER, T. **Semantic Versioning 2.0.0**. 2010. Disponível em: https://semver.org. Acesso em: 2026.

SLACK, N.; BRANDON-JONES, A.; JOHNSTON, R. **Administração da Produção**. 8. ed. São Paulo: Atlas, 2018.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

VERNON, V. **Implementing Domain-Driven Design**. Boston: Addison-Wesley, 2013.

VOLLMANN, T. E.; BERRY, W. L.; WHYBARK, D. C.; JACOBS, F. R. **Manufacturing Planning and Control for Supply Chain Management**. 5. ed. New York: McGraw-Hill, 2005.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. W3C Recommendation, 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 2026.